

Smart Visual Alarm System: On-Device Person Detection with MQTT and Telegram Notifications

Erneyei Zoltán

Matriculation No.: 265056

Project Technical Report

Repository: [ernyei13/iot-project-pedestrian-detector](https://github.com/ernyei13/iot-project-pedestrian-detector)

Date: 2026-06-12

Abstract—In this project I built a Smart Visual Alarm System with the ESP32-S3-EYE board. The device uses its onboard camera to detect a person locally, confirms the detection over several frames, then sends an alarm event through MQTT. A small Python backend subscribes to the MQTT topic and forwards the alert to a Telegram bot. The prototype was tested with the board LCD showing a person bounding box and a Telegram message showing the event details. Overall, the system demonstrates the main requirements from the assignment: embedded person detection, confirmed alarm generation, MQTT communication, and Telegram notification. Reliability, false-alarm behavior, and latency are discussed as evaluation criteria for the prototype.

Index Terms—ESP32-S3-EYE, embedded vision, MQTT, Telegram Bot API, IoT alarm, person detection.

I. INTRODUCTION

The project brief asks for a low-cost visual alarm system that detects the presence of a person using the ESP32-S3-EYE onboard camera. The device must perform people detection on the embedded board, generate an alarm

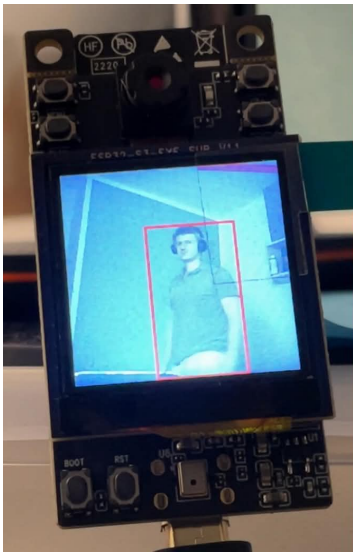
only after a valid detection is confirmed, publish an event through MQTT, and trigger a Telegram notification through a simple Python backend.

The implementation follows that scope directly. Camera frames remain local to the ESP32-S3-EYE, while downstream components receive only alarm metadata: event identifier, device identifier, timestamp, label, confidence, confirmation count, and bounding-box coordinates. This design keeps network payloads small and avoids sending raw camera images to a cloud vision service.

II. SYSTEM ARCHITECTURE

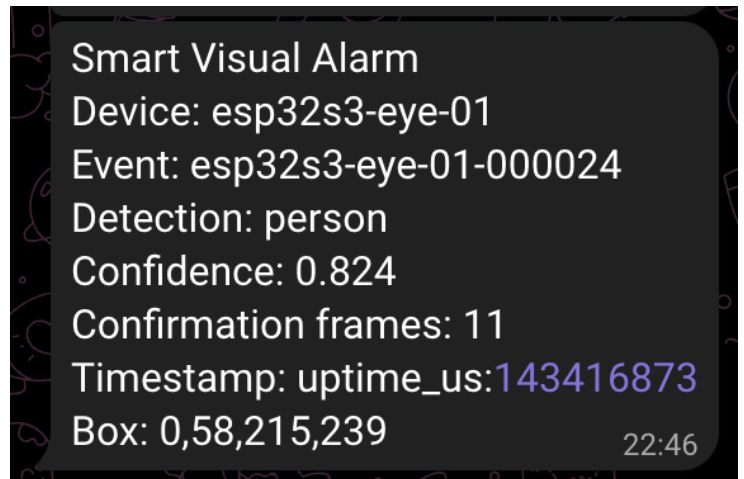
The system is organized as a four-layer pipeline. The camera and LCD provide sensing and local operator feedback. The detector and alarm policy convert frame-level results into events. MQTT carries the event metadata. The Python backend validates and forwards the notification to Telegram.

- **Camera/display:** captures RGB565 frames and shows the LCD preview with a person bounding box.



(a)

ESP32-S3-EYE LCD detection.



(b) Telegram MQTT alert.

Fig. 1. The system running example demonstration.

- **Detection policy:** uses ESP-WHO/ESP-DL pedestrian detection with threshold, confirmation frames, and cooldown.
- **MQTT transport:** publishes JSON events to `smart-visual-alarm/events`; local Mosquitto can act as broker.
- **Backend:** validates payloads, formats the alert, applies a cooldown, and calls Telegram `sendMessage`.

The end-to-end data path is: camera frame to on-device person detection to confirmation policy to MQTT JSON to Python subscriber to Telegram Bot message.

III. FIRMWARE IMPLEMENTATION

The ESP-IDF firmware initializes status hardware, starts the optional LCD preview, attempts Wi-Fi and MQTT startup, then runs the detection loop. The frame capture pipeline sends camera frames to the detector task. For each cycle, the application selects the strongest person candidate, checks the configured confidence threshold, counts consecutive positive frames, and suppresses repeated alerts during the cooldown interval.

The AI model is Espressif’s `pedestrian_detect` component, version 0.3.0, using the `PICO_S8_V1` pedestrian detector stored as `pedestrian_detect_pico_s8_v1.espd1`. In this ESP32-S3 build the model file is 435,328 bytes and is embedded in flash read-only data. The model input is $224 \times 224 \times 3$; Espressif reports approximately 9.2 ms preprocessing, 118.3 ms model inference, and 2.4 ms postprocessing on ESP32-S3. At runtime, RGB565 camera frames are passed to the ESP-WHO detection task, which calls the ESP-DL model locally and returns bounding boxes plus confidence scores. The public component documentation does not state the original training dataset, so the project treats it as a vendor-provided pretrained pedestrian model rather than a model trained inside this project.

The main alarm controls are exposed through Kconfig: detection score threshold, confirmation frames, alert cooldown, MQTT broker URI, and MQTT topic. The MQTT startup path is defensive. If Wi-Fi or broker settings are absent, the detector still runs and logs alarm JSON over serial. If network settings are present, the firmware starts station mode, waits for an IP address, initializes ESP-MQTT, and publishes events to the configured topic.

IV. BACKEND IMPLEMENTATION

The backend is intentionally small and auditable. It loads local environment variables, connects to the configured MQTT broker, subscribes to the alarm topic, validates incoming JSON, formats a readable alert, and sends it through Telegram. Dry-run mode logs the formatted alert instead of calling Telegram, which makes the backend testable before live credentials are available.

Validation rejects malformed JSON, missing required fields, non-string identifiers, invalid confidence values, and

invalid confirmation counts. The final Telegram message preserves the metadata needed for a useful alarm: device ID, event ID, label, confidence, confirmation count, device timestamp, and bounding box.

V. EVENT FORMAT AND REPRODUCIBILITY

The MQTT event is intentionally compact and text based. JSON was selected because it is easy to inspect in serial logs, publish with common MQTT tools, and parse from Python without a custom binary decoder. The event payload includes:

- **event_id:** unique alarm identifier for logs, screenshots, and debugging.
- **device_id:** source board identity, e.g. `esp32s3-eye-01`.
- **timestamp:** device uptime timestamp when wall-clock time is not configured.
- **confidence:** detector confidence used to explain alert quality.
- **confirmation_frames:** consecutive positive frames behind the alarm.
- **box:** bounding-box coordinates for later review and presentation evidence.

The event ID combines the device identity with a monotonic counter, making repeated alarms distinguishable even when a wall-clock timestamp is unavailable.

The repository keeps reproducibility notes close to the code: ESP-IDF v5.5.2 is the selected firmware toolchain, local Mosquitto is the recommended broker for demonstration, and secrets are expected to live only in ignored local configuration files. This separation is important for submission quality because the code remains publishable while the demo can still use real Wi-Fi and Telegram credentials locally.

VI. VERIFICATION

The prototype was checked at firmware, backend, and integration levels. ESP-IDF v5.5.2 builds the firmware image for `esp32s3`. The Python backend compiles and runs in a clean dependency environment. A sample MQTT event published to the local broker is received by the backend and forwarded to Telegram.

- **Firmware build:** ESP-IDF produced a flashable application binary for ESP32-S3.
- **Backend syntax:** Python compilation succeeded for the MQTT-to-Telegram bridge.
- **MQTT bridge:** a sample event reached the backend and was forwarded to Telegram.
- **LCD evidence:** the board display shows a person candidate with bounding box.
- **Telegram evidence:** the alert message contains the expected alarm metadata.

VII. INTEGRATED RUN EVIDENCE

Fig. 1 captures the two most important evidence artifacts. The device photograph shows that the ESP32-S3-EYE displays a person candidate with a red bounding

box. The Telegram screenshot shows the corresponding style of notification produced by the backend from alarm metadata.

The Telegram message includes `esp32s3-eye-01` as the device, event `esp32s3-eye-01-000024`, detection label `person`, confidence `0.824`, `11` confirmation frames, timestamp `uptime_us:143416873`, and bounding box `0,58,215,239`. This is the right level of detail for an alarm prototype: it is readable by a user and preserves enough metadata to debug the model and transport path.

VIII. EVALUATION SUMMARY

The prototype satisfies the main functional requirements from the brief. It captures camera frames, performs person detection using the ESP32 vision framework, generates an alarm only after confirmation, publishes structured MQTT-compatible metadata, and forwards a readable message to Telegram.

- **Reliability:** person detection is visible on the board LCD during the demonstrated run.
- **False alarms:** confirmation frames and cooldown reduce single-frame noise; extended no-person testing was outside this prototype run.
- **Latency:** Telegram delivery is demonstrated; synchronized timing measurements were not part of this report.
- **Repeatability:** broker, backend, and build steps are documented; credentials remain local.

The evidence is strongest for functional integration. The statistical evaluation is limited because the report focuses on a compact demonstration rather than a repeated experiment over many scenes. The relevant test cases for this type of system include empty scene, person entering, person standing, moving non-person object, low light, backlight, and partial occlusion.

IX. EVALUATION PROTOCOL

The evaluation focuses on functional proof of the end-to-end pipeline. The report documents a confirmed person detection on the ESP32-S3-EYE display, a structured MQTT-style event, and the matching Telegram notification. The evaluation criteria are detection result, broker delivery, and notification arrival time.

X. RISKS AND MITIGATIONS

Lighting and camera angle may change detection confidence, so final placement should be tuned after scene testing. Wi-Fi or broker setup can fail during a live demo, so local Mosquitto, laptop IP verification, and serial JSON logging should remain available. Telegram credentials must be stored only in ignored local files, and bot tokens should be rotated after testing. The report therefore frames the system as a functional prototype rather than production security equipment.

XI. REPRODUCIBLE DEMONSTRATION PROCEDURE

The most reliable demonstration path uses the laptop as the MQTT broker and Telegram bridge host. First, start Mosquitto with the repository LAN configuration so the broker listens on the network interface, not only on localhost. Second, start the Python backend with live Telegram credentials stored in an ignored local environment file. Third, flash or boot the ESP32-S3-EYE firmware with local Wi-Fi credentials and the broker URI set to the laptop address. Finally, place a person in front of the camera and verify three artifacts: the LCD bounding box, the broker publish log, and the Telegram message.

This procedure is intentionally simple because classroom demonstrations fail most often at operational boundaries rather than inside the detector. The serial fallback is also useful: if Wi-Fi or broker discovery fails, the firmware still logs the alarm JSON, proving that detection and event generation continue to run on device. The backend can then be demonstrated separately with the sample event payload.

XII. SUBMISSION PACKAGE

The repository is organized as a compact submittable code package. The `firmware/` directory contains the ESP-IDF project and ESP-WHO-derived components. The `backend/` directory contains the MQTT-to-Telegram bridge, sample payload, requirements file, and Mosquitto LAN configuration. The repository root contains the setup `README.md`, this report as `report.pdf`, and the presentation as `presentation.pptx`.

XIII. DEPLOYMENT CONSTRAINTS

The prototype depends on placement and lighting. The ESP32-S3-EYE camera should face the monitored area directly, avoid strong backlight when possible, and be mounted so the person occupies enough pixels for the pedestrian detector. The confirmation-frame rule helps reduce transient noise, but it also introduces a small delay before alerting. A lower threshold may increase sensitivity but can also increase false alarms.

XIV. CONCLUSION

The Smart Visual Alarm System meets the core brief: it acquires frames from the ESP32-S3-EYE camera, performs on-device person detection, confirms valid detections, emits structured alarm metadata, and forwards alerts to Telegram through a Python backend. The LCD and Telegram images make the report evidence-based rather than purely descriptive. The submitted system demonstrates the complete assigned workflow from embedded detection to user notification.

REFERENCES

- [1] Project repository: `README.md`, `firmware/`, `backend/`, `report.pdf`, and `presentation.pptx`.
- [2] Firmware source: `firmware/main/app_main.cpp`, `firmware/main/alarm_mqtt.cpp`, frame capture, detection, and LCD preview modules.

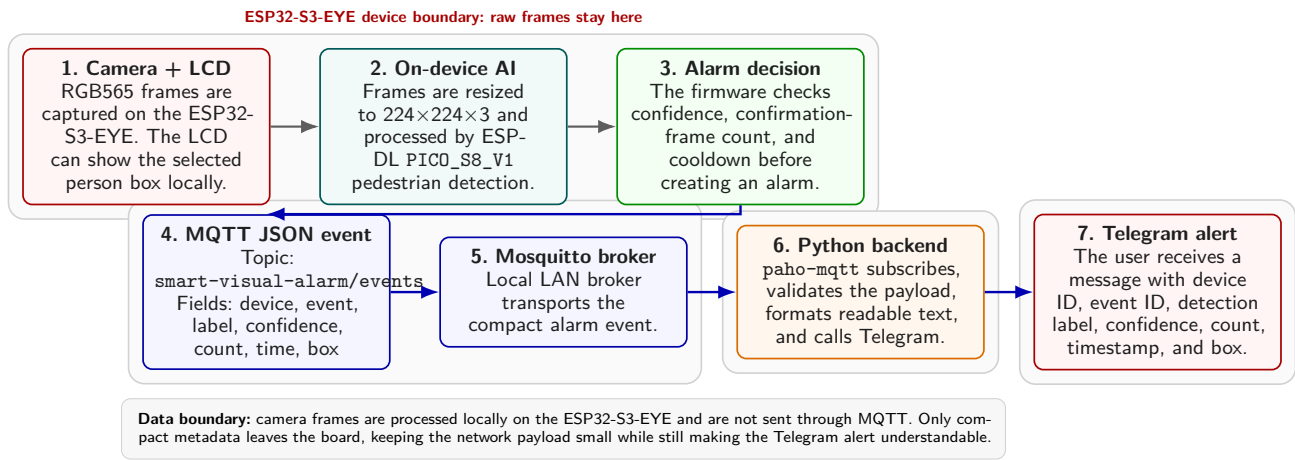


Fig. 2. End-to-end system flow from local camera capture to the Telegram notification. The important design boundary is that raw image frames stay on the ESP32-S3-EYE; MQTT and Telegram carry only alarm metadata.

- [3] Backend source: `backend/mqtt_to_telegram.py`, `backend` README, Mosquitto LAN config, and sample event payload.
- [4] Espressif, ESP-IDF Programming Guide v5.5.2 for ESP32-S3.
- [5] Espressif, ESP-WHO repository, ESP32-S3-EYE examples, and `pedestrian_detect` model component documentation.
- [6] MQTT.org, MQTT: The Standard for IoT Messaging.
- [7] Telegram, Bot API `sendMessage` documentation.